KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Appointment Scheduling and Reminder System

Submitted by

RATISH V

B.Sc. Information Technology

Under the Guidance of

DHILEEBAN A

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: 2025 – 2026

OCTOBER 2026

Semester: II

DECLARATION

I hereby declare that the project entitled **“Appointment Scheduling and Reminder System”**, submitted in partial fulfillment of the requirements for the course **JAVA PROGRAMMING**, is a record of my work carried out under the guidance of **DHILEEBAN A**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or similar title at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

October 2026
Coimbatore

Signature of the Student: _____

CERTIFICATE

This is to certify that the project entitled **“Appointment Scheduling and Reminder System”** is a bona fide work carried out by **RATISH V**, B.Sc. Information Technology, KG College of Arts & Science, in partial fulfillment of the requirements for the course **JAVA PROGRAMMING**.

This project has been completed under my supervision and guidance during the academic period 2026.

Place: COIMBATORE
Date: _____

Submitted for the Viva-Voce Examination held on _____

Guide	Head of the Department
Technical Trainer, KGCAS	Technical Head, KGCAS
_____	_____

TABLE OF CONTENTS

S. No.	Topics	Page No.
1	Introduction	5
1.1	Objective	6
1.2	Overview	7
1.3	Features	8
2	Program Analysis	9
2.1	Problem Definition	10
2.2	Project Overview	11
2.3	Scope and Limitations	12
3	System Design	13
3.1	Modules	14
4	Methodology	15
4.1	Problem Analysis	16
4.2	System Design	17
4.3	Program Development	18
4.4	Testing and Validation	19
4.5	Result and Documentation	20
4.6	Algorithm	21
4.7	Flowchart	22
5	Source Code	23
6	Input and Output Screens	24
7	Discussion and Conclusion	25
7.1	Discussion	26
7.2	Conclusion	27
8	References	28

ABSTRACT

The Appointment Scheduling and Reminder System is a Java-based application designed to help users create, view, search, and manage appointments in an organized way. Manual appointment tracking can lead to missed meetings, forgotten visits, and confusion about dates and times. This project provides a simple menu-driven solution for recording appointment details and displaying reminders.

The system allows a user to add an appointment with a title, person or service name, date, time, and contact details. Users can view scheduled appointments, search for an appointment, cancel an appointment, and check for upcoming reminders.

The application demonstrates Core Java concepts such as classes, objects, methods, ArrayList, loops, conditional statements, and date-time handling. The current educational version stores appointments in memory while the program is running. Database storage and automatic SMS or email notifications can be added in future versions.

The main objective is to improve appointment organization, reduce missed appointments, and provide clear feedback for each operation.

1. INTRODUCTION

Appointments are used in hospitals, clinics, educational institutions, offices, and service centres. When appointments are maintained only in notebooks or informal messages, users may forget scheduled times, lose details, or accidentally arrange two appointments at the same time.

The Appointment Scheduling and Reminder System is a Java application that allows users to record appointment details, view appointments, search for a record, cancel an appointment, and check for upcoming reminders. Its menu-driven interface makes the program easy to operate.

1.1 Objective

The main objective is to develop a simple appointment management application using Core Java.

- Create and store appointment details in a structured format.
- Display all scheduled appointments clearly.
- Search using appointment ID.
- Cancel an appointment when required.
- Check upcoming appointments and display reminder messages.
- Demonstrate object-oriented programming and date-time handling.

1.2 Overview

The program displays a menu and asks the user to choose an operation. Appointment data is represented as objects and stored in an ArrayList during execution. The reminder option compares appointment times with the current time.

1.3 Features

- Add appointment details.
- View and search appointments.
- Cancel an appointment.
- Validate date and time input.
- Display reminders for appointments within the next 60 minutes.
- Show useful messages for invalid input.

2. PROGRAM ANALYSIS

2.1 Problem Definition

Manual appointment management can be unreliable when there are many appointments or when schedules change frequently. Information may be recorded inconsistently, reminders may be missed, and it may take time to find a particular record.

- Appointments may be forgotten without reminders.
- Searching paper records can take time.
- Cancellation details may not be updated clearly.
- Date, time, or contact details may be missing.
- Users may have no single view of upcoming appointments.

2.2 Project Overview

The application is implemented as a Java console program. An Appointment class represents each record, while AppointmentService manages adding, listing, searching, cancelling, and checking reminders.

2.3 Scope and Limitations

Scope: suitable for a small personal appointment list or classroom demonstration and can be adapted for clinics, offices, tutors, and service centres.

Limitations: records are stored in memory and are lost when the program closes. Reminders appear only when the user checks them while the program is running. The basic version does not send SMS, email, or push notifications.

3. SYSTEM DESIGN

System design describes the main components and how information moves through the application. The user selects an operation, enters details, the service performs the requested action, and a result message is displayed.

3.1 Modules

1. Menu Display Module: displays options and reads the user's choice.

2. Appointment Creation Module: accepts a title, name, contact, date, and time; validates required fields and assigns an ID.

3. Appointment Listing Module: displays all stored appointment records.

4. Search and Cancellation Module: locates an appointment by ID and cancels it when requested.

5. Reminder Module: compares appointment times with the current time and displays upcoming reminders.

6. Exit Module: closes the application safely.

System Flow

User → Main Menu → Appointment Service → Appointment List → Result or Reminder Message.

4. METHODOLOGY

A structured development process was followed to plan, implement, and test the application. The stages are problem analysis, system design, program development, testing, and documentation.

4.1 Problem Analysis

The system requirements are to record appointment details, display and search records, support cancellation, and show reminders for upcoming appointments.

4.2 System Design

The program is divided into Main, Appointment, and AppointmentService components. Main handles menu and input; Appointment stores one appointment; AppointmentService performs scheduling operations.

4.3 Program Development

Core Java is used. Classes and methods organize the code, an ArrayList stores appointments, loops repeat the menu, and conditional statements process each selected option. Date-time parsing and exception handling help manage invalid input.

4.4 TESTING AND VALIDATION

Testing checks whether the system behaves as expected for valid and invalid inputs. Each operation should be tested separately and then tested as part of a complete program run.

- Valid appointment details → appointment saved and ID displayed.
- Empty title or name → validation message shown.
- View option → all stored appointments displayed.
- Existing ID → matching appointment displayed.
- Unknown ID → appointment not found message.
- Invalid date format → error message and program continues.
- Reminder option → appointments in the next 60 minutes displayed.

4.5 Result and Documentation

The expected result is a console program that can create, display, search, and cancel appointments and show reminders for appointments that are near. Documentation describes the purpose, design, algorithm, source code, and sample console interaction.

4.6 ALGORITHM AND 4.7 FLOWCHART

Algorithm for the Appointment Scheduling and Reminder System:

- Start the program.
- Create the appointment service and input scanner.
- Display the main menu.
- Read the user's selected option.
- If Add is selected, read and validate details, assign an ID, and store the appointment.
- If View is selected, display all appointments.
- If Search is selected, locate the record by ID.
- If Cancel is selected, update the appointment status.
- If Reminder is selected, compare appointment time with current time and display upcoming reminders.
- Repeat until Exit is selected.
- Close resources and end the program.

Flowchart

START ↓ Display Menu ↓ Read User Choice ↓ Perform Add / View / Search / Cancel / Reminder ↓ Exit selected? If No, return to menu; if Yes, END.

5. SOURCE CODE — APPOINTMENT.JAVA

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class Appointment {
    private final int id;
    private final String title, personName, contact;
    private final LocalDateTime dateTime;
    private String status;

    public Appointment(int id, String title, String personName,
                       String contact, LocalDateTime dateTime) {
        this.id = id;
        this.title = title;
        this.personName = personName;
        this.contact = contact;
        this.dateTime = dateTime;
        this.status = "Scheduled";
    }
    public int getId() { return id; }
    public String getTitle() { return title; }
    public String getPersonName() { return personName; }
    public String getContact() { return contact; }
    public LocalDateTime getDateTime() { return dateTime; }
    public String getStatus() { return status; }
    public void cancel() { status = "Cancelled"; }

    public String toString() {
        DateTimeFormatter f =
            DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm");
        return "ID: " + id + " | " + title + " | " + personName
            + " | " + dateTime.format(f) + " | " + status;
    }
}
```

5. SOURCE CODE — SERVICE CLASS

```
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

public class AppointmentService {
    private final List<Appointment> appointments = new ArrayList<>();
    private int nextId = 1;

    public Appointment add(String title, String person, String contact,
                           LocalDateTime dateTime) {
        Appointment a = new Appointment(nextId++, title, person,
                                        contact, dateTime);
        appointments.add(a);
        return a;
    }
    public List<Appointment> getAll() {
        return new ArrayList<>(appointments);
    }
    public Appointment findById(int id) {
        for (Appointment a : appointments)
            if (a.getId() == id) return a;
        return null;
    }
    public boolean cancel(int id) {
        Appointment a = findById(id);
        if (a == null || a.getStatus().equals("Cancelled")) return false;
        a.cancel();
        return true;
    }
    public List<Appointment> upcoming(int minutes) {
        LocalDateTime now = LocalDateTime.now();
        LocalDateTime limit = now.plusMinutes(minutes);
        List<Appointment> result = new ArrayList<>();
        for (Appointment a : appointments) {
            if (a.getStatus().equals("Scheduled")
                && !a.getDateTime().isBefore(now)
                && !a.getDateTime().isAfter(limit))
                result.add(a);
        }
        return result;
    }
}
```

5. SOURCE CODE — MAIN CLASS (PART 1)

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.List;
import java.util.Scanner;

public class Main {
    private static final Scanner sc = new Scanner(System.in);
    private static final AppointmentService service =
        new AppointmentService();
    private static final DateTimeFormatter INPUT_FORMAT =
        DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm");

    public static void main(String[] args) {
        boolean running = true;
        while (running) {
            System.out.println("\\n=== APPOINTMENT SYSTEM ===");
            System.out.println("1. Add appointment");
            System.out.println("2. View appointments");
            System.out.println("3. Search appointment");
            System.out.println("4. Cancel appointment");
            System.out.println("5. Check reminders (next 60 minutes)");
            System.out.println("6. Exit");
            System.out.print("Choose option: ");
            String choice = sc.nextLine().trim();
            switch (choice) {
                case "1": addAppointment(); break;
                case "2": viewAppointments(); break;
                case "3": searchAppointment(); break;
                case "4": cancelAppointment(); break;
                case "5": checkReminders(); break;
                case "6": running = false; break;
                default: System.out.println("Invalid option.");
            }
        }
        System.out.println("Thank you for using the system.");
        sc.close();
    }
```

5. SOURCE CODE — MAIN CLASS (PART 2)

```
private static void addAppointment() {
    System.out.print("Appointment title: ");
    String title = sc.nextLine().trim();
    System.out.print("Person / service name: ");
    String person = sc.nextLine().trim();
    System.out.print("Contact details: ");
    String contact = sc.nextLine().trim();
    System.out.print("Date and time (dd-MM-yyyy HH:mm): ");
    String dateText = sc.nextLine().trim();

    if (title.isEmpty() || person.isEmpty()) {
        System.out.println("Title and name are required.");
        return;
    }
    try {
        LocalDateTime dt = LocalDateTime.parse(dateText, INPUT_FORMAT);
        if (dt.isBefore(LocalDateTime.now())) {
            System.out.println("Choose a future date and time.");
            return;
        }
        Appointment a = service.add(title, person, contact, dt);
        System.out.println("Appointment added. ID: " + a.getId());
    } catch (DateTimeParseException ex) {
        System.out.println("Invalid date/time format.");
    }
}

private static void viewAppointments() {
    List<Appointment> all = service.getAll();
    if (all.isEmpty()) {
        System.out.println("No appointments found.");
        return;
    }
    for (Appointment a : all) System.out.println(a);
}

private static void searchAppointment() {
    System.out.print("Enter appointment ID: ");
    try {
        int id = Integer.parseInt(sc.nextLine().trim());
        Appointment a = service.findById(id);
        System.out.println(a == null ? "Appointment not found." : a);
    } catch (NumberFormatException ex) {
        System.out.println("Enter a valid numeric ID.");
    }
}
```

5. SOURCE CODE — MAIN CLASS (PART 3)

```
    private static void cancelAppointment() {
        System.out.print("Enter appointment ID to cancel: ");
        try {
            int id = Integer.parseInt(sc.nextLine().trim());
            if (service.cancel(id))
                System.out.println("Appointment cancelled.");
            else
                System.out.println("Not found or already cancelled.");
        } catch (NumberFormatException ex) {
            System.out.println("Enter a valid numeric ID.");
        }
    }

    private static void checkReminders() {
        List<Appointment> list = service.upcoming(60);
        if (list.isEmpty()) {
            System.out.println("No appointments in the next 60 minutes.");
            return;
        }
        System.out.println("--- UPCOMING APPOINTMENT REMINDERS ---");
        for (Appointment a : list)
            System.out.println("Reminder: " + a);
    }
}
```

6. INPUT AND OUTPUT SCREENS

The following sample console output illustrates how the application can be used. The actual output depends on the appointment details entered.

```
=== APPOINTMENT SYSTEM ===
1. Add appointment
2. View appointments
3. Search appointment
4. Cancel appointment
5. Check reminders (next 60 minutes)
6. Exit
Choose option: 1
Appointment title: Project Review
Person / service name: Ratish
Contact details: 9876543210
Date and time (dd-MM-yyyy HH:mm): 28-10-2026 10:30
Appointment added. ID: 1

Choose option: 2
ID: 1 | Project Review | Ratish | 28-10-2026 10:30 | Scheduled

Choose option: 4
Enter appointment ID to cancel: 1
Appointment cancelled.

Choose option: 6
Thank you for using the system.
```

If screenshots are required, run the program in an IDE or terminal and insert genuine screenshots of appointment creation, listing, reminder checking, and cancellation.

7. DISCUSSION AND CONCLUSION

7.1 Discussion

The system demonstrates how a Java application can organize appointment details and make them easier to access. The menu-driven structure supports adding, viewing, searching, and cancelling appointments and checking upcoming reminders.

The Appointment class represents each appointment as an object. AppointmentService separates operations from menu code, while ArrayList stores records during execution. Java date-time classes help parse and compare scheduled times.

Testing should include valid appointments, missing fields, invalid date formats, past dates, unknown IDs, cancellation, and reminder checking. The current version stores data in memory, so records are not retained after the program exits. Reminders are displayed when the user checks them; this version does not run an automatic notification service.

7.2 Conclusion

The Appointment Scheduling and Reminder System demonstrates Core Java concepts in a practical scheduling application. It provides basic operations for creating, viewing, searching, and cancelling appointments, as well as identifying appointments that are coming up soon.

Future improvements can include database connectivity, automatic email or SMS notifications, appointment rescheduling, prevention of overlapping bookings, user login, calendar views, and a graphical user interface.

8. REFERENCES

The following resources can be consulted for Java programming and date-time handling used in this project.

Books

- Herbert Schildt – Java: The Complete Reference.

Websites

- Oracle Java Documentation – https://docs.oracle.com/en/java/
- GeeksforGeeks – https://www.geeksforgeeks.org/
- W3Schools Java Tutorial – https://www.w3schools.com/java/
- Java Date and Time API – https://docs.oracle.com/javase/8/docs/api/java/time/package-summary.html

— END OF REPORT —